Software Requirements Specification Template

Software Engineering

The following annotated template shall be used to complete the Software Requirements Specification (SRS) assignment.

Template Usage:
Text contained within angle brackets ('<', '>') shall be replaced by your project-specific information and/or details. For example, <Project Name> will be replaced with either 'Smart Home' or 'Sensor Network'.

Italicized text is included to briefly annotate the purpose of each section within this template. This text should not appear in the final version of your submitted SRS.

This cover page is not a part of the final template and should be removed before your SRS is submitted.

Movie Theater Ticketing

Software Requirements Specification

Version 3.0

03/19/26

Group #12
Adyant Manchanda, Daynelle Lorin Sadangsal, Samantha Manuyag

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.

Fall 2023

Revision History

Date	Description	Author	Comments
02/19/26	Version 1.0	Group-12	Requirements Specification
03/05/26	Version 2.0	Group-12	Software Design Specification
03/19/26	Version 3.0	Group-12	Test Plan

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

REVISION HISTORY ..... **II**

DOCUMENT APPROVAL ..... **II**

1. INTRODUCTION **1**

1.1 PURPOSE 1
1.2 SCOPE 1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 2
1.4 REFERENCES 2
1.5 OVERVIEW 2

2. GENERAL DESCRIPTION **2**

2.1 PRODUCT PERSPECTIVE 3
2.2 PRODUCT FUNCTIONS 3
2.3 USER CHARACTERISTICS 3
2.4 GENERAL CONSTRAINTS 3
2.5 ASSUMPTIONS AND DEPENDENCIES 4

3. SPECIFIC REQUIREMENTS **5**

3.1 EXTERNAL INTERFACE REQUIREMENTS 5
3.1.1 User Interfaces *5*
3.1.2 Hardware Interfaces *5*
3.1.3 Software Interfaces *5*
3.1.4 Communications Interfaces *5*
3.2 FUNCTIONAL REQUIREMENTS 6
3.2.1 Managing User Accounts *6*
3.2.2 Movie Searching and Filtering *6*
3.2.3 Theater Seating Options *6*
3.2.4 Price Displays *6*
3.2.5 Transactions *6*
3.2.6 Payments *7*
3.2.7 Refunds *7*
3.2.8 Tickets *7*
3.3 USE CASES 7
3.3.1 Use Case #1 - Purchase Movie Ticket *7*
3.3.2 Use Case #2 - Request Refund *9*
3.3.3 Use Case #3 ..... *10*
3.4 CLASSES / OBJECTS 11
3.4.1 <Class / Object #1> *11*
3.4.2 <Class / Object #2> *11*
3.5 NON-FUNCTIONAL REQUIREMENTS 11
3.5.1 Performance *12*
3.5.2 Reliability *12*
3.5.3 Availability *12*
3.5.4 Security *12*
3.5.5 Maintainability *12*

3.5.6 Portability *13*
3.6 SOFTWARE DESIGN REQUIREMENTS 14
3.6.1 System Description *14*
3.6.2 Software Architecture Overview *14*
3.6.3 Development Plan and Timeline *18*
3.8 LOGICAL DATABASE REQUIREMENTS 20
3.9 OTHER REQUIREMENTS 20

4. ANALYSIS MODELS 20

4.1 SEQUENCE DIAGRAMS 20
4.3 DATA FLOW DIAGRAMS (DFD) 20
4.2 STATE-TRANSITION DIAGRAMS (STD) 20

5. CHANGE MANAGEMENT PROCESS 20

A. APPENDICES 20

A.1 APPENDIX 1 20
A.2 APPENDIX 2 20

1. Introduction

The introduction of the Software Requirements Specification (SRS) provides the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the SRS. This document aims to give an in-depth insight into the Movie Theater Ticketing (MTT) system in detail. The detailed requirements of the Movie Theater Ticketing system are provided in this document.

1.1 Purpose

The purpose of this document is to provide a detailed overview of the MTT software system, designed to allow customers to browse movies, select showtimes and seats, and purchase tickets through a secure and efficient online website. This document defines the system's functional and non-functional requirements needed for its design and implementation, with respect to customers and clients. Therefore, this document serves as a primary reference for the system's development and expectations.

1.2 Scope

Primarily, the scope pertains to the development and implementation of the MTT System, a browser-based application designed to support online ticket sales. The system will allow customers to browse movies, view showtimes, select seats, and purchase digital tickets securely and efficiently. Additionally, the system is based on a relational database that manages movie listings, theater locations, scheduling, availability, and transaction records.

1.3 Definitions, Acronyms, and Abbreviations

SRS	Software Requirements Specification
MTT	Movie Theater Ticketing
Deluxe Theater	Theater with assigned seating (Total: 75 seats)
Regular Theater	Theater with open seating (Total: 150 seats)
PCI-DSS	Payment Card Industry Data Security Standard
FTC	Federal Trade Commission
CFPB	Consumer Financial Protection Bureau
CCPA	California Consumer Privacy Act
CPRA	California Privacy Rights Act

1.4 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

1.5 Overview

The remaining sections of this document provide an overview of the MTT system requirements, including the characteristics of its users, system environment, and functionality of the product. Section 2 gives a general description of the project. Section 3 defines the functional and non-functional requirements, client design constraints, and user assumptions. Section 4 describes the analysis models used to validate the specific requirements of the MTT system. Section 5 clarifies the management process if changes to this SRS are needed.

2. General Description

This section of the SRS describes the general factors that affect the MTT system and its requirements. This section does not state specific and detailed requirements, but rather clarifies the requirements defined in Section 3.

2.1 Product Perspective

The MTT system is a web-based application designed to support online and in-person kiosk ticket sales. It operates within a server where customers can access the system through web browsers on desktops, mobile devices, and theater kiosks. This server connects to a database that stores and manages the following data: movie details, theater description, customer information, reservation details, and transaction records.

This system also uses external systems such as third-party payment services, email providers, authentication services for customer login verification, and hardware, such as ticket scanners, card readers, receipt/ticket printers, and kiosks.

The MTT system supports two theater designs: Deluxe Theater and Regular Theater. The system intends to support or replace traditional box office operations through a secure and efficient digital ticketing service.

2.2 Product Functions

The MTT system shall provide the following major functions:

1. **Account Management**: Includes customer account registration, login functionality, password and email verification, and account information.
2. **Movie Search and Selection**: Includes a display of available movies with description and poster, search functionality by title, and filtered search by genre, theater location and showtime.
3. **Seat Selection**: Includes visual to seat layout, indication of available seats, and ticket quantity selection (maximum of 20 per showtime).
4. **Checkout**: Includes display of price per ticket and calculation of total cost including fees and taxes.
5. **Payment Processing**: Includes handling of transactions with multiple supported payment methods and error handling for incomplete or failed transactions.
6. **Ticket Generation**: Includes generation of digital tickets with a unique barcode, email delivery of confirmation, and printable options.

2.3 User Characteristics

This section describes the general characteristics of the eventual users of the MTT system that will affect the specific requirements. General customers are individuals who purchase

movie tickets. These users are expected to have basic knowledge with web browsing, a supported payment method, and access to an email account. Theater Staff Members use the system through ticket validation tools such as QR code scanners and kiosks. These users are expected to have basic technical training in ticketing operations and customer service.

2.4 General Constraints

Regulatory Policies

1. The system must comply with PCI-DSS for secure payment processing.
2. The system must comply with Consumer Protection and Transparency regulations enforced by FTC and CFPB.
3. The system must comply with Consumer Privacy of Personal Data enforced by CCPA/CPRA.
4. The system must follow applicable taxation.

Hardware Limits

1. The system must operate on theater kiosk hardware, such as touchscreen devices, card readers, receipt printers, and QR code scanners.
2. The system must operate on standard mobile and desktop browsers.
3. The system must not exceed the hardware memory or processing of kiosks.

Interfaces to Other Applications

1. The system must integrate third-party payment processors.
2. The system must integrate email service providers.
3. The system must integrate authentication services.
4. The system must connect to the theater database.

Parallel Operation

1. The system must support multiple users accessing the system online and through kiosks.
2. The system must prevent double booking during seat selection.

Audit Functions

1. The system must update transaction records.
2. All purchases and refunds must be recorded with timestamps.

Control Functions

1. The system must allow authorized administrators to manually manage movie listings, showtimes, pricing, and seating.

Higher-Order Language Requirements

1. The system must be developed using industry-standard programming and technologies.

Signal Handshake Protocols

1. The system must use secure HTTPS protocols.
2. The hardware must communicate with kiosk hardware.

Reliability Requirements

1. The system must ensure real-time seat availability.
2. The system must maintain consistent and real-time data across kiosks and online interfaces.

Criticality of the Application

1. The system must ensure stability in the event of a system failure or during peak periods, including weekends, holidays, or movie releases.

Safety and Security Considerations

1. The system must protect customer data using encryption and security services.
2. The system shall require higher security and authorization of administrative functions.
3. The system shall generate unique QR codes for each issued ticket to prevent duplication or fraud.

2.5 Assumptions and Dependencies

Assumptions

1. Offered movies, showtimes, and availability are updated in real-time.
2. Users have reliable internet access when purchasing tickets.
3. Theater kiosks must maintain stable connections to the system.

Dependencies

1. The system depends on web-hosting infrastructure.
2. The system depends on email service providers for providing ticket delivery and confirmation.
3. The system depends on third-party payment processing services for transactions.

3. Specific Requirements

This section details the requirements for the Movie Ticketing System (MTT). These include external interface requirements, which describe how the MTT system interacts with the user and other systems, as well as functional requirements that define the system's primary features.

3.1 External Interface Requirements

3.1.1 User Interfaces

1. The system shall be accessible to any customer without requiring account registration.
2. The system shall be accessible through the online website.
3. The system shall be accessible in-person at a theater kiosk.
4. The system shall allow the user to go back and forth between pages.
5. The system shall be compatible with all screen configurations and ratios.

3.1.2 Hardware Interfaces

1. The software shall be supported at in-person movie theater kiosks with touch-screen displays.
2. The system shall be compatible with QR code scanners in the theater for ticket validation.
3. The system shall support credit and debit card readers attached to the kiosk.
4. The system shall support receipt printers connected to theater kiosks.

3.1.3 Software Interfaces

1. The software shall be able to run on iOS and Android mobile devices.
2. The system shall connect to an email service provider to handle account notifications, payment confirmations, and refund requests.
3. The system shall connect to a payment processing system to handle all transactions.
4. The system shall connect to an authentication system for user login and multi-factor authentication.
5. The system shall connect to movie and showtime databases to access movies listings and showtime schedules.

3.1.4 Communications Interfaces

1. The system shall use HTTPS protocol for communication between databases and the Internet.

3.2 Functional Requirements

This section includes core features and behaviors of the MTT.

3.2.1 Managing User Accounts

1. The system shall allow the user to create an optional account using a valid email address.
2. The system shall allow the user to log in and log out.
3. The system shall display an error message if the user's email address or password is incorrect.
4. The system shall allow the user to change their registered email.
5. The system shall allow the user to reset their password.
6. The system shall send a password-reset link to the user's registered email.
7. The system shall send the user a confirmation email after the user successfully resets their password.
8. The system shall offer optional multi-factor verification via email verification codes.

3.2.2 Movie Searching and Filtering

1. The system shall allow the user to search for specific movies.
2. The system shall allow the user to filter specific theaters, genres, and showtimes.
3. The system shall provide the user with a scrollable list of movies and their posters.
4. The system shall provide the user with a list of theaters with showtimes of the selected movie.
5. The system shall provide the user with available showtimes for the selected theater.

3.2.3 Theater Seating Options

1. The system shall allow the user to see the layout of the theater seats.
2. The system shall display available seats in green and unavailable seats in red.
3. The system shall display an error message if a user selects a seat that is already taken.
4. The system shall provide the user with a dropdown menu of all other available seats in the same row as the selected seat.
5. The system shall limit the user to a maximum of twenty tickets per showtime.
6. The system shall display an error message if the user attempts to purchase over the ticket limit.

3.2.4 Price Displays

1. The system shall provide the user with price information for one ticket.
2. The system shall provide the user with the total price of multiple tickets.

3.2.5 Transactions

1. The system shall allow the user to confirm their purchase and proceed to the checkout page.
2. The system shall provide a final checkout page that displays the total price including tax.
3. The system shall allow the user to cancel their purchase.

3.2.6 Payments

1. The system shall allow the user to choose their preferred payment method.

2. The system shall allow the user to manually input payment information during transactions.
3. The system shall allow users with an account to save their payment information.
4. The system shall display an error message if the user leaves a payment information box blank.
5. The system shall display an error message if the payment method fails.

3.2.7 Refunds

1. The system shall allow the user to request a refund for any eligible-owned ticket(s).
2. The system shall prevent refund requests submitted less than two hours before the scheduled showtime.
3. The system shall display an error message when a user attends to request a refund for a non-eligible ticket.
4. The system shall issue a refund to the user within an hour of its initiation.
5. The system shall provide the user with a confirmation email of the refund.
6. The system shall issue refunds to the user's original payment method.

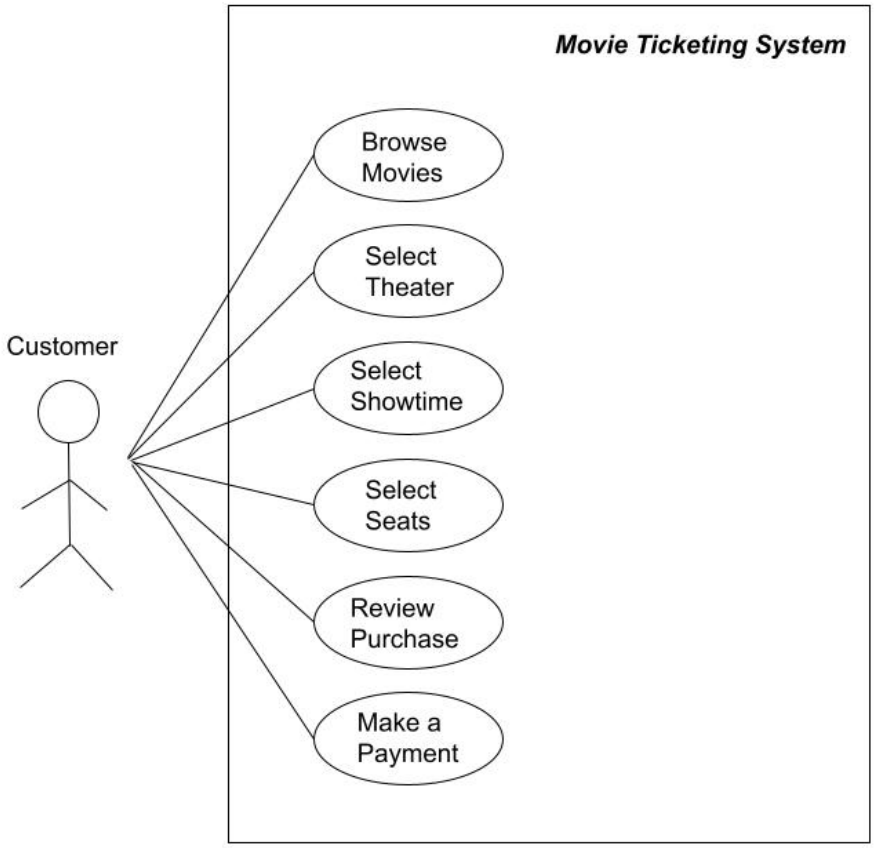
3.2.8 Tickets

1. The system shall send a QR code for the user's tickets to their registered email.
2. The system shall prompt the user to input an email if they don't have a registered account.
3. The system shall give the user the option to download a printable version of their ticket(s).
4. The system shall allow the user to transfer tickets to another registered user.
5. The system shall transfer tickets via email.
6. The system shall allow transferred tickets to be printed or scanned as a QR code.

3.3 Use Cases

3.3.1 Use Case #1 – Purchase Movie Ticket

Use Case: A customer visits the movie ticketing website and purchases a ticket for their selected movie at a chosen theater and showtime.



Primary Actor: Customer
Basic Flow:
1. The customer visits the movie ticketing website.
2. The system displays all available movies across all available theaters.
3. The customer finds and chooses the movie they want to watch.
4. The system displays available theaters.
5. The customer chooses a theater.
6. The system displays available showtimes for the chosen movie and theater.
7. The customer chooses showtime.
8. The system displays the theater room layout.
9. The customer chooses their preferred seat.
10. The system displays the total price of the selected ticket(s), including tax.
11. The customer confirms their selection.
12. The system navigates to the checkout page.
13. The customer inputs their payment information and makes their payment.
14. The customer enters a valid email address if one is not already stored in the system.
15. The system sends a confirmation receipt and ticket QR code to the customer's email address.

Alternative Flows:

3a. The customer changes the movie, theater, or showtime before choosing a seat.
    3a1. The system updates the movie, theater, and showtime availability.
9a. The customer chooses a different seat.
    9a1. The system updates the newly selected seat.
    9a2. The system updates the total price if needed.
9b. The customer selects a different number of seats.
    9b1. The system updates the number of seats.
    9b2. The system updates the total price.
11a. The customer cancels their purchase.
    11a1. The system returns the user to the movie selection page.

Pre-conditions:

1. The customer has access to the system.
2. The system is online and operational.

Post-conditions:

1. If the Basic Flow is completed, the customer has successfully bought their ticket(s).
2. If Alternative Flow #4 occurs, the purchase is canceled and no tickets are issued.

3.3.2 Use Case #2 – Request Refund

Use Case: A customer submits a refund request for one or more purchased tickets, and the system verifies before issuing a refund.
Primary Actors: Customer
Supporting Actors: Payment processing service, Email service provider
Basic Flow:
1. The customer visits the movie ticketing website.
2. The customer logs into their account.
3. The customer navigates to the Purchased Tickets page.
4. The system displays the customer's purchases of tickets and indicates which tickets are refundable.
5. The customer selects one or more tickets and clicks the "Request Refund" button.
6. The system verifies if the selected tickets meet refund eligibility requirements.
7. The system displays the refund summary including ticket details, refund amount, and payment method.
8. The customer confirms the refund request.
9. The system sends the refund request to the payment processing service.
10. The payment processor confirms and accepts the refund.
11. The system updates the database's transaction records with a refund and timestamp.
12. The system sends a refund confirmation email to the customer.

Alternative Flows:
6a. Tickets do not meet refund eligibility requirements.

6a1. If the showtime is less than two hours before, the system denies the refund request and displays an error message.

6a2. If the ticket has already been used or refunded, the system denies the refund request and displays an error message.

8a. The customer cancels the refund.

8a1. If the customer does not confirm the refund request and cancels, the system terminates the refund process and returns to the Purchased Tickets page.

10a. The payment processing service rejects the refund.

10a1. The system displays an error message, and the refund is not issued.

Pre-Conditions:

1. The customer has access to the system.
2. The customer has a registered account.
2. The customer has previously purchased tickets.
3. The system is online and operational.
4. The system is connected to the payment processing service.

Post-Conditions:

1. If the Basic Flow is completed, the customer has successfully refunded the tickets, the refund is issued to the payment method, the system database is updated, and the customer receives a confirmation email.
2. If any of the Alternative Flows occur, no refund is issued, the system displays an error message, and the refund request is terminated.

3.3.3 Use Case #3

Theater administrators alternating the movies that are being displayed on the platform.

Primary Actor: Theater Administrator

Basic Flow:

1. The administrator logs into the system with admin credentials
2. The System displays the admin dashboard with a menu to manage the movies out on display
3. The administration chooses to add a new movie
4. The system prompts the admin to put to the movie title, genre, rating, description, and all showtimes
5. The administrator confirms the new movies details
6. The system updates the movie listings and displays a confirmation popup
7. The system shows the updated movie list

Alternative Flows:

1. Edit and Existing Movie:
   1.1 The administrator chooses an existing movie from the list to edit its current details such as show times
   1.2 The system allows the admin to update movie information such as description genre or times.
   1.3 The system saves the changes and confirms the update

2.Delete a movie:
   2.1 The administrator selects a movie and removes it from the system
   2.2 The system prompts for confirmation before removal, and it updates the movie listings after the changes

Pre-Conditions:
1. The administrator is logged into the admin portal
2. The movie details for the new entry are available
Post-Conditions:
1. The movie list is updated with the edited, new, or deleted movie
2. Notifications are sent to users to inform about movie changes

3.4 Classes / Objects

3.4.1.1 Attributes

movie_title: String – The title of the movie.

showtime: DateTime – The date and time of the movie showing.

theater: String – The type of theater (e.g., Deluxe or Regular).

seat: String – The selected seat.

price: Float – The price of the ticket.

payment_status: String – The current payment status (e.g., Paid, Pending, Failed).

ticket_id: String – A unique identifier for the ticket

3.4.1.2 Functions

generateTicket(): Generates a digital ticket with a unique ticket ID and barcode.

sendTicket(): Sends the ticket to the user's registered email.

cancelTicket(): Cancels a reserved ticket before showtime, making the seat available.

updatePrice(new_price: Float): Updates the ticket price if there is a change in the ticket pricing.

refundTicket(): Issues a refund if eligible under the refund policy.

3.4.2 User

3.4.2.1 Attributes

user_id: String – A unique identifier for the user.

name: String – The full name of the user.

email: String – The user's email address.

password: String – The hashed password for the user.

payment_info: String – A secure token or representation of the user's payment method.

purchase_history: List[MovieTicket] – A list of tickets the user has purchased.

3.4.2.2 Functions

login(email: String, password: String): Authenticates the user with their credentials.

logout(): Logs the user out of the system.

updateAccountInfo(new_email: String, new_password: String): Allows the user to update their account details.

resetPassword(): Sends a reset link to the user's registered email.

viewPurchaseHistory(): Displays a list of all purchased tickets by the user

3.4.3 PaymentProcessor

3.4.3.1 Attributes

payment_id: String – Unique identifier for each transaction.

payment_status: String – Current status of the payment (e.g., Success, Failed).

payment_method: String – The payment method used (e.g., Credit Card, PayPal).

amount: Float – The total amount of the transaction.

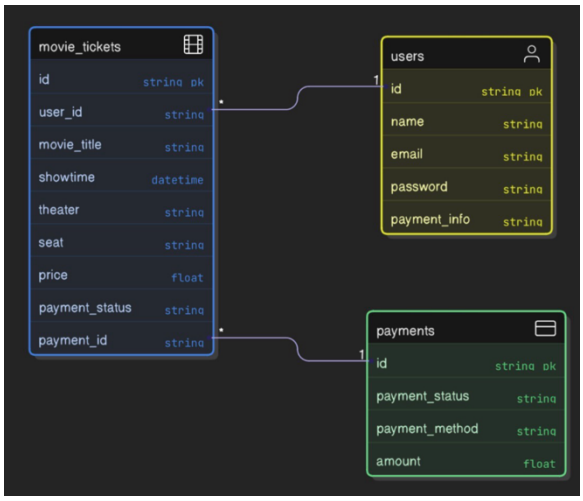
3.4.3.2 Functions

processPayment(): Processes the payment using the chosen payment method.

refundPayment(): Refunds the payment to the user's original payment method.

validatePaymentInfo(): Verifies the validity of the provided payment details before processing.

UML Diagram :



3.5 Non-Functional Requirements

3.5.1 Performance

1. The system shall support up to 5000 concurrent users during peak hours (e.g., weekends, holidays)

2. Transactions (movie ticket purchases) should be processed within 2 seconds.

3.5.2 Reliability

1. The system shall have an uptime of 99.99% over a monthly period, allowing for minimal downtime (e.g., 4.38 minutes per month).

2. In case of failure, the system should ensure automatic rollback to avoid data loss during transaction failures.

3.5.3 Availability

1.The system shall be accessible 24/7, with scheduled downtime for maintenance on Sundays between 2:00 AM to 4:00 AM UTC.

2. The system should support auto-scaling during peak hours to ensure consistent availability.

3.5.4 Security

1. All user data, including personal information and payment details, shall be encrypted using AES-256 encryption.

2.User passwords must be stored as hashes using bcrypt or similar hashing algorithms.

3.The system shall comply with PCI-DSS standards for secure payment processing.

4.Two-factor authentication (2FA) shall be required for administrators accessing the backend.

3.5.5 Maintainability

1. The system should allow for easy updates and patches with zero downtime. It should support continuous integration and deployment (CI/CD) pipelines.

2.The system should include automated tests to verify functionality after each deployment.

3.Code should be modular and well-documented to enable easy updates and enhancements by future developers.

3.5.6 Portability

1. The system should be compatible with major web browsers (Chrome, Firefox, Safari, Edge) and mobile platforms (iOS and Android).

2.The system should operate on the following devices:

2.1 Kiosk hardware with touchscreen displays.

2.2 Web browsers for both desktop and mobile.

3. The system should support cloud-based hosting with the ability to deploy across multiple regions for faster access.

3.6 Software Design Requirements

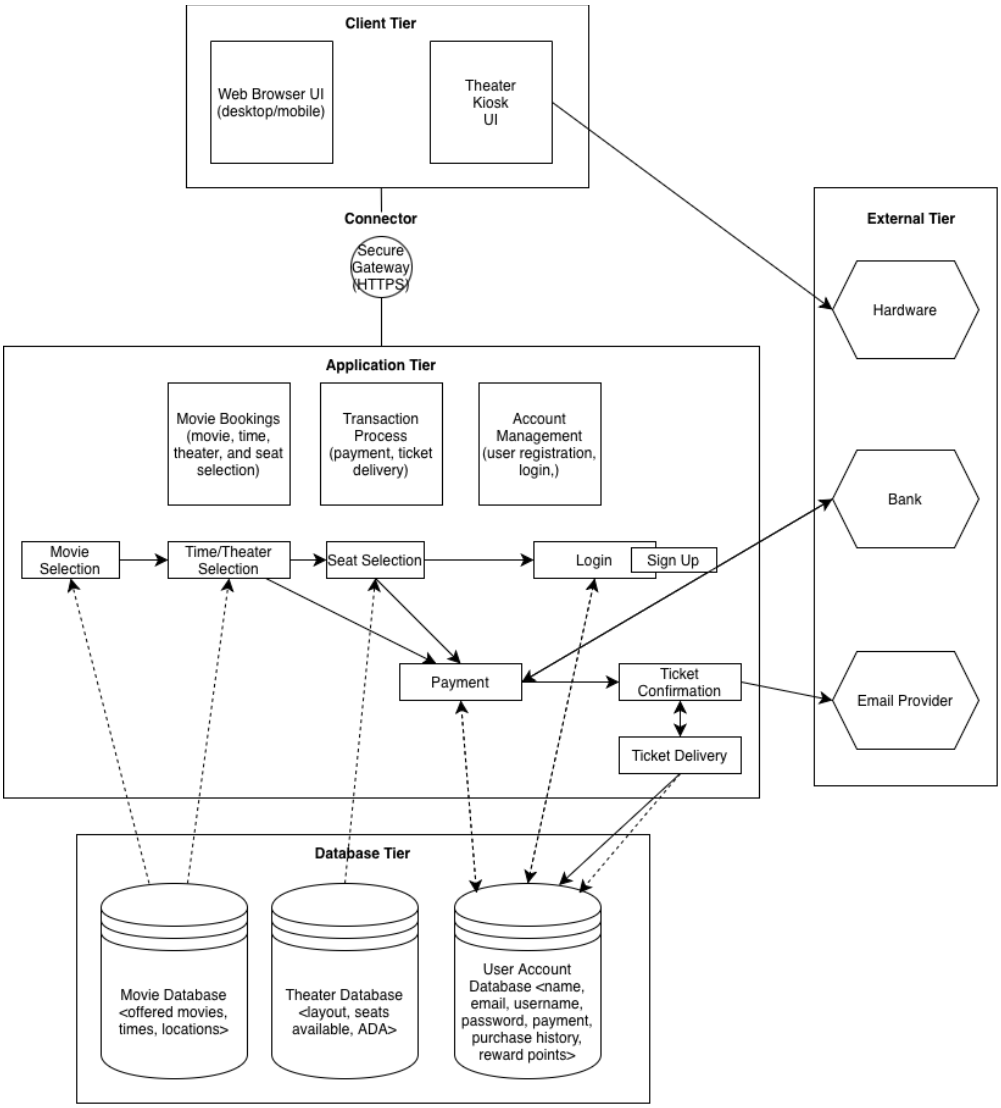
3.6.1 System Description

The Movie Theater Ticketing (MTT) system is a browser-based platform designed for developers to implement a scalable system for 20 theaters in San Diego. The system is also designed to support up to 10 million users by using a partitioned database and queueing system for high-demand movie showings, while maintaining a secure, HTTPS-encrypted connector between the client and the application server. This software system will manage three main databases: Movies, Theaters, and User Accounts. Additionally, the software must interface with physical hardware such as theater kiosks, QR code scanners, card readers, and receipt printers.

3.6.2 Software Architecture Overview

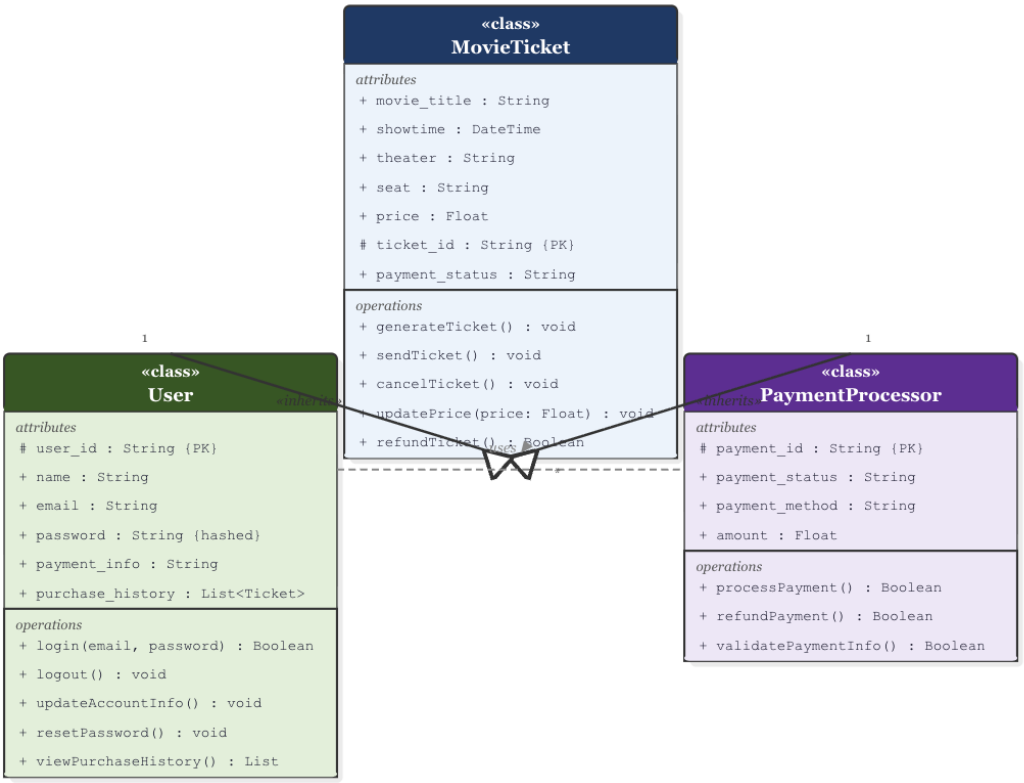
3.6.2.1 Architectural diagram of all major components

Based on the system requirements, the MTT system follows a multilevel client-server architecture that consists of the following tier components: Client, Application, Database, and External.



3.6.2.2 UML Class Diagram

The UML class diagram below defines the core classes of the movie ticket system.



3.6.2.3 Description of classes

The Movie Theater Ticketing system is structured around three classes: MovieTicket, User Class, and PaymentProccesor.

MovieTicket Class:

The MovieTicket Class is the foundational data object of the MTT system. It represents a single ticket purchased by a customer for a specific movie, showtime, theater, and seat. This class is responsible for storing all ticket related data and providing core operations such as ticket generation, delivery, cancellation, pricing and refund processing. Every transaction in the system is tied to one or more MovieTicket objects.

User Class:

The UserClass represents a registered customer account within the Movie Theater Ticketing system. It extends the Movie Ticket class and stores personal account information including authentication credentials and payment data. The User class manages the full account life cycle. From registration and login to password resets and purchase history. Users may optionally save payment information to streamline future purchases.

PaymentProcessor Class:

The PaymentProcessor Class handles all financial transactions within the Movie Theater Ticketing system. It extends the Movie Ticket class and manages payment execution, validation and refunds. This class interfaces with third-party payment services to securely process transactions across multiple payment methods. It ensures compliance with PCI-DSS standards by delegating sensitive operations to external processors and storing only transaction metadata internally.

3.6.2.4 Description of attributes

Movie Tickets Class

Attribute	Type	Description
movie_title	string	Title of the user's chosen movie
showtime	datetime	Specific show time for the selected movie
theater	string	The user's chosen theater for the selected movie
seat	string	Selected seat assigned to the user's ticket
price	float	Price of the selected seat(s)
ticket_id	string pk	The user's ticket identifier

User Class (Sub-class of Movie Tickets Class)

Attribute	Type	Description
id	string pk	User's unique account identifier
name	string	The user's full name
email	string	The user's registered email address for primary login
password	string	User's password for authentication
payment_info	string	Stores the user's payment information (credit card details, PayPal account, etc.)
purchase_history	list	List of a user's entire purchase history

Payments Class (Sub-class of Movie Tickets Class)

Attribute	Type	Description
id	string pk	User's unique transaction identifier
payment_status	string	Pending, successful, or failed payment
payment_method	string	Payment method used (credit card, PayPal, Zelle, etc.)
amount	float	Total price of the user's purchase

3.6.2.5 Description of operations

Ticket Operations

Operation	Description
generateTicket()	Creates a ticket for the user's selected seat, showtime, and movie.
sendTicket()	Transfers an owned ticket to another device or registered account holder.
cancelTicket()	Cancels a ticket prior to the completion of the final transaction.
updatePrice(new_price: Float)	Recalculates the total price when tickets are added or cancelled, and when tax is applied.
refundTicket()	Issues a refund for an eligible ticket and updates the user's previous ticket status.

User Account Operations

Operation	Description
login(email: String, password: String)	Validates the user's login credentials and grants access to their account.
logout()	Ends the user's session on the website.
updateAccountInfo(new_email: String, new_password: String)	Updates the user's account details (email or password).
resetPassword()	Initiates a password rest and updates the user's password.

viewPurchaseHistory()	Displays all past purchases made on the user's account.

Payment Operations

Operation	Description
validatePaymentInfo()	Validates the user's payment method details, such as credit card or PayPal information.
processPayment()	Process the user's transaction and complete the payment.
refundPayment()	Issues the refund to the user's original payment method.

3.6.3 Development Plan and Timeline

This section gives a brief breakdown of the different development tasks and team member responsibilities.

3.6.3.1 Partitioning of Tasks

1. Separating the Software into Modules
2. Scope and Purpose of the MTT Software
3. General Requirements: Functional and Non-Function Requirements
4. UI/UX Designs
5. UML and SWA Diagrams

3.6.3.2 Team Member Responsibilities

Software Engineers:
Adyant Manchanda

1. Non-functional Requirements
2. UML Diagrams
3. Classes/Objects
4. Writing Test Cases

Daynelle Lorin Sadangsal

1. System Introduction
2. System Overview
3. General System Description
4. Architectural Diagram

5. Writing Test Cases

Samantha Manuyag

1. External Interface Requirements
    a. Hardware Interface
    b. Software Interface
    c. User Interface
    d. Communication Interface
2. Functional Requirements
3. Description of Attributes and Operations
4. Writing Test Cases

4. Test Plan

4.1 Overview

The Movie Theater Ticketing (MTT) test plan utilizes a verification and validation strategy, ensuring that the implementation aligns with the design specifications, UML class diagram, overall system requirements, while also meeting the customer-client needs. Testing is divided into three granularities. Firstly, unit testing verifies individual methods and classes. Secondly, functional testing verifies specific features and requirements. Lastly, system testing verifies the system workflow.

GitHub Repository:

4.2 Test Scope

The primary system components being tested include account management, movie browsing, seat selection, payment processing, ticket generation, and refund handling. In addition to functional features, the scope of testing also covers the system architecture tiers: Presentation, Application, Database, and External.

In-Scope:

- User account features such as login, logout, password reset, and purchase history
- Movie browsing features such as search and filtering by title, genre, theater, and showtime
- Seat selection and availability validation
- Ticket quantity limits
- Price Calculation
- Payment process, such as validation of payment and handling of successful and failed transactions

- Ticket generation, including QR code and email delivery
- Refund eligibility and processing

Out-of-Scope:

- Performance testing of third-party payment systems
- Physical hardware testing
- Security of external services and infrastructure
- Implementation of third-party APIs (banking and email providers)

4.3 Testing Process

The MTT systems testing will use the following three different levels of testing:

Unit Testing: Unit testing targets individual functions and methods within the three classes: MovieTicket, User, and PaymentProcessor. Each operation is testing in isolation using mock inputs to simulate external dependencies such as the payment processor and email provider. Unit Tests verify that each function behaves correctly for valid inputs, invalid inputs, and edge cases such as boundary conditions.

Functional Testing: Functional testing verifies that the MTT system's features satisfy the requirements stated in section 3.2. Test cases work on the complete functional workflows such as a user browsing movies, picking a seat, completing a transaction, and receiving confirmation of their ticket. Each test case maps directly to one or more functional requirements to ensure all features are fully tested.

System Testing: System testing validates the end-to-end behavior of the fully integrated system, including interactions between the web front-end, application serve, database, and external services. System tests cover complete user journeys such as purchasing a ticket, refunding requests, and account management.

4.4 Testing Environment

The following environment will be used to execute the test cases defined in this test plan:
Hardware:

Standard desktop and laptop computers running Windows or macOS
Mobile devices running iOS and Android for browser compatibility testing
Simulated kiosk environment using a touchscreen-enabled tablet device

Software & Browsers:

Web browsers: Google Chrome 120+, Mozilla Firefox 120+, Safari 17+, Microsoft Edge 120+
Mobile browsers on iOS (Safari) and Android (Chrome)

Back-End & Database:

Application server running in a local or cloud-based staging environment
Relational database (MySQL or PostgreSQL) pre-populated with test movie listings, showtimes, theater layouts, and user accounts

External Services (Sandboxed):

Payment processing: Stripe test API sandbox (no real transactions)
Email delivery: Mail trap or similar sandboxed email service to capture outbound emails
Authentication: Test instance of the authentication service

Testing Tools:

Manual testing performed by team members through the web browser

4.5 Tested Items

User Authentication

Movie Browsing

Seat Selection
Ticket Purchases
- Ticket generation
- Confirmation displays through selection and payment process
-

Refunds
- Check for eligibility of a ticket for refund
- Refund execution to the correct account

Device Compatibility
- Mobile
- Desktop

- Kiosk
- Tablet

Account Notifications
- Login verification
- Email of payment confirmation
- Delivery of ticket QR code(s)

4.6 Testing Constraints

4.7 Test Cases

Group 12 Test Cases

4.7 GitHub

GitHub Repository

3.7 Design Constraints

Specify design constrains imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2